

Step 1: Create your Lambda function

Lambda > Functions > Create function

Create function info

Choose one of the following options to create your function.

Author from scratch

Start with a simple Hello World example.

Use a blueprint

Build a Lambda application from sample code and configuration presets for common use cases.

Container image

Select a container image to deploy for your function.

Basic information

Function name

Enter a name that describes the purpose of your function.

FUNCTION_NAME

Function name must be 1 to 64 characters, must be unique to the Region, and can't include spaces. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (_).

Runtime

Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.

Python 3.14

Architecture

Choose the instruction set architecture you want for your function code.

arm64

x86_64

Permissions

By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. You can customize this default role later when adding triggers.

Change default execution role

Additional configurations

Use additional configurations to set up networking, security, and governance for your function. These settings help secure and customize your Lambda function deployment.

Cancel Create function

Lambda > Functions > FUNCTION_NAME

Successfully created the function FUNCTION_NAME. You can now change its code and configuration. To invoke your function with a test event, choose "Test".

FUNCTION_NAME

Throttle Copy ARN Actions

Export to Infrastructure Composer Download

Function overview

Diagram Template

FUNCTION_NAME

Layers (0)

+ Add trigger

+ Add destination

Description

Last modified

Function ARN

Function URL

Code

Test

Monitor

Configuration

Aliases

Versions

Code source

Open in Visual Studio Code l2 Upload from

EXPLORER

FUNCTION_NAME

lambda_function.py

lambda_function.py

1 import json

2

3 def lambda_handler(event, context):

4 # TODO Implement

5 return (

6 'statusCode': 200,

7 'body': json.dumps('Hello from Lambda!')

8)

9

Step 2: Create Amazon API Gateway

API Gateway > APIs > Create API > Create REST API

Create REST API Info

API details

☒ **New API**
Create a new REST API.

☐ **Clone existing API**
Create a copy of an API in this AWS account.

☐ **Import API**
Import an API from an OpenAPI definition.

☐ **Example API**
Learn about API Gateway with an example API.

API name
user

Description - optional
Users API

API endpoint type
Regional APIs are deployed in the current AWS Region. Edge-optimized APIs route requests to the nearest CloudFront Point of Presence. Private APIs are only accessible from VPCs.
Regional

Security policy - new Info
Transport Layer Security (TLS) protects data in transit between a client and server. The security policy also determines the cipher suite options that clients can use with your API.
Choose a security policy

IP address type Info
Select the type of IP addresses that can invoke the default endpoint for your API.
☒ **IPv4**
Supports only edge-optimized and Regional API endpoint types.
☐ **Dualstack**
Supports all API endpoint types.

[Cancel](#) [Create API](#)

Create resource

Resource details

☒ **Proxy resource** Info
Proxy resources handle requests to all sub-resources. To create a proxy resource use a path parameter that ends with a plus sign, for example {proxy+}.

Resource path
/
Resource name
get-user

☐ **CORS (Cross Origin Resource Sharing)** Info
Create an OPTIONS method that allows all origins, all methods, and several common headers.

[Cancel](#) [Create resource](#)

Create method

Method details

Method type
GET

Integration type

☒ **Lambda function**
Integrate your API with a Lambda function.


☐ **HTTP**
Integrate with an existing HTTP endpoint.


☐ **Mock**
Generate a response based on API Gateway mappings and transformations.


☐ **AWS service**
Integrate with an AWS Service.


☐ **VPC link**
Integrate with a resource that isn't accessible over the public internet.


☒ **Lambda proxy integration**
Send the request to your Lambda function as a structured event.

Response transfer mode Info
☒ **Buffered**
Wait to receive the complete response before beginning transmission.
☐ **Stream**
Send portions of the response without waiting for the complete response.

Lambda function
Provide the Lambda function name or alias. You can also provide an ARN from another account.
us-west-1

 **Grant API Gateway permission to invoke your Lambda function**
When you save your changes, API Gateway updates your Lambda function's resource-based policy to allow this API to invoke it.

Integration timeout Info
By default, you can enter an integration timeout of 50 - 29,000 milliseconds. You can use Service Quotas to raise the integration timeout to greater than 29,000 ms
29000

Successfully created deployment for user. This deployment is active for Test.

0 0 4 0 0 0

Stages

Test

Stage actions Create stage

Stage details Info Edit

Stage name
Test

Cache cluster Info
☐ Inactive

Default method-level caching
☐ Inactive

Invoke URL
<https://ew0at6n534.execute-api.us-west-1.amazonaws.com/Test>

Active deployment
yixm3p on February 27, 2026, 14:49 (UTC-08:00)

Rate Info
10000

Burst Info
5000

Web ACL
-

Client certificate
-

Logs and tracing Info Edit

CloudWatch logs
☐ Inactive

X-Ray tracing
☐ Inactive

Detailed metrics
☐ Inactive

Data tracing
☐ Inactive

Step 3: Setup API keys for accessing API Gateway

Create API key Info

API key details

Name

client key

Description - optional

API key

☒ Auto generate

☐ Custom

Step 4: Create usage plan for stage and specify allowed API keys.

✔ Successfully created API key 'client key'.

Create usage plan [Info](#)

Usage plan details
Usage plans specify the number or rate of requests that your API accepts from a client. Associate an API key with a usage plan to track the requests your API receives.

Name

Description - optional

☒ **Throttling**
Limit the rate that users can call your API.

Rate

Burst

☐ **Quota**
Turn on quotas to limit the number of requests a user can make to your API in a given time period.

Requests
Enter the total number of requests that a user can make in the time period you select in the dropdown list.

✔ Successfully added stage "Test" to usage plan. 0 0 0 4 0 0 0

client key

Last updated February 27, 2026, 15:03 (UTC-08:00) [Refresh](#) [Edit](#) [Delete](#) [Add to usage plan](#)

API key details	Status ✔ Active
ID k7y9mseziF	API key [icon] Show
Description -	
Creation date February 27, 2026, 14:54 (UTC-08:00)	

[Associated usage plans](#) [Tags](#)

Usage plans (1)				Last updated February 27, 2026, 15:03 (UTC-08:00)	Refresh Remove
Usage plan ID	Name	Associated APIs	Associated stages		
☐ x32ge6	plan	user	Test		

[Create resource](#)

/

/get-user

GET

/get-user - GET - Method execution

ARN
[icon] arn:aws:execute-api:us-west-1:204312534875:ew0at6n534/*/GET/get-user

Resource ID
2y2mpo

Client

Method request

Integration request

Method response

Integration response
Proxy integration

Lambda integration

[Method request](#) [Integration request](#) [Integration response](#) [Method response](#) [Test](#)

Method request settings [Edit](#)

Authorization NONE	API key required True
Request validator None	SDK operation name Generated based on method and path

Successfully created deployment for user. This deployment is active for Test.

000600

Stages

Test

Stage details

Stage name

Test

Rate

10000

Cache cluster

Inactive

Burst

5000

Default method-level caching

Inactive

Web ACL

-

Client certificate

-

Invoke URL

https://ew0at6n534.execute-api.us-west-1.amazonaws.com/Test

Active deployment

gy3xb7 on February 27, 2026, 15:11 (UTC-08:00)

Edit

Last login: Wed Feb 25 14:56:08 on ttys001

jane@Janes-MacBook-Air ~ % curl -i \

-H "x-api-key: yQYzKDKvCl8XemFb8tqA5wOF6mCwwSv1FW3b5M8g" \

["https://ew0at6n534.execute-api.us-west-1.amazonaws.com/Test/get-user"

HTTP/2 200

date: Fri, 27 Feb 2026 23:13:42 GMT

content-type: application/json

content-length: 20

x-amzn-requestid: b8823161-8725-4a0c-9aae-59e40a32b624

x-amz-apigw-id: ZdQ9-FLnyK4Ee\$w=

x-amzn-trace-id: Root=1-69a22525-460a690f543b21007f533851;Parent=4aba7e255ecf32fa;Sampled=0;Lineage=1:8b803c5d:0

"Hello from Lambda!"

HTTP/2 403

date: Fri, 27 Feb 2026 23:15:31 GMT

content-type: application/json

content-length: 23

x-amzn-requestid: 2966526b-dac3-417b-a7e5-67edc1ede6dd

x-amzn-errortype: ForbiddenException

x-amz-apigw-id: ZdrPDEMfyK4E0tw=

jane@Janes-MacBook-Air ~ %

jane@Janes-MacBook-Air ~ % curl -i "https://ew0at6n534.execute-api.us-west-1.amazonaws.com/Test/get-user"